Unit 1
BASICS OF OPERATING SYSTEM

1.1. Introduction to operating system:

- ✓ An Operating System (OS) is an interface between a computer user and computer hardware.
- ✓ An operating system is a program that acts as an interface between the user and the computer hardware and controls the execution of all kinds of programs.
- ✓ The software that contains the core components of the operating system is called the **kernel**.

Objectives of OS:

The primary goals of an operating system are as follows:

- **Convenience** – An operating system improves the use of a machine. Operating systems enable users to get started on the things they wish to complete quickly without having to cope with the stress of first configuring the system.
- **Efficiency** – An operating system enables the efficient use of resources. This is due to less time spent configuring the system.
- **Ability to evolve** – An operating system should be designed in such a way that it allows for the effective development, testing, and introduction of new features without interfering with service.
- **Management of system resources** – It guarantees that resources are shared fairly among various processes and users.

1.2. Architecture of Operating System:

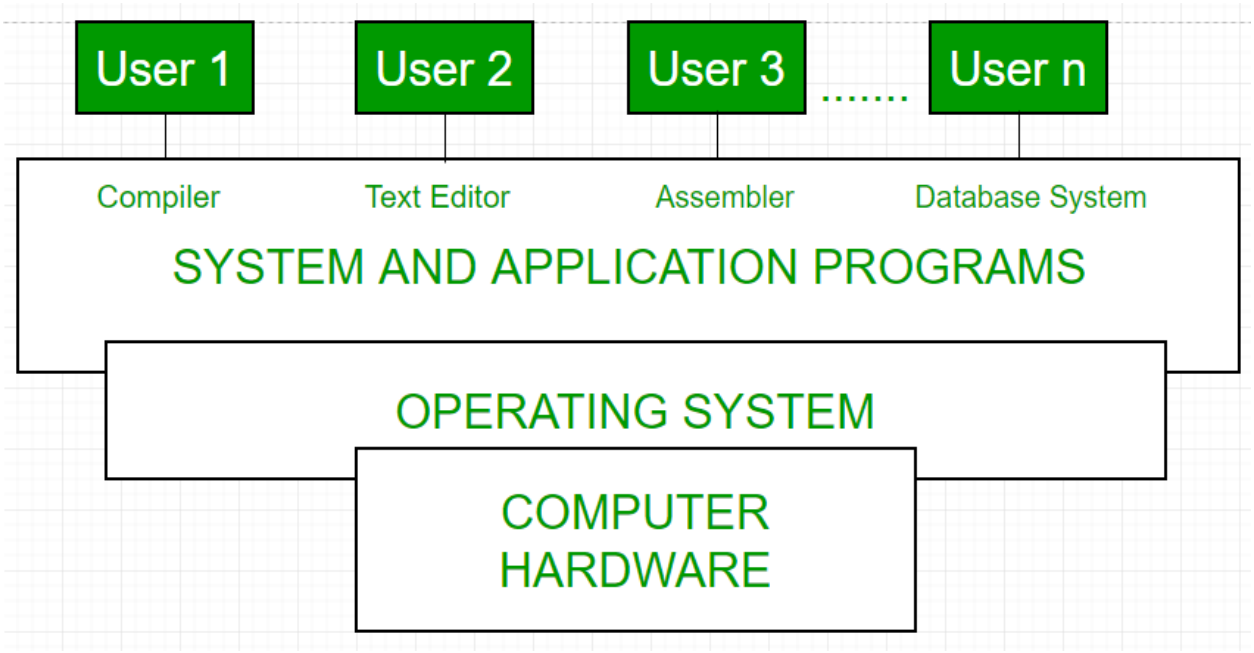


Fig: Architecture of Operating System

1.3. Functions of Operating System:

1. Memory Management:

- It is the management of the main or primary memory.
- The operating system Allocates and deallocates the memory.
- Keeps a record of which part of primary memory is used by whom and how much.
- Distributes the memory while multiprocessing.

2. Processor Management/Scheduling:

- Processor management is an execution unit in which a program operates.
- The operating system determines the status of the processor and processes, selects a job and its processor, allocates the processor to the process, and de-allocates the processor after the process is completed.

3. Device Management:

- An Operating System manages device communication via their respective drivers.
- It Keeps tracks of all devices.
- Decides which process gets the device when and for how much time.
- Allocates the device in the efficient way.

4. File Management:

- The operating system manages resource allocation and de-allocation.
- It specifies which process receives the file and for how long.
- It also keeps track of information, location, uses, status, and so on.
- These groupings of resources are referred to as file systems.

5. Storage Management:

- Storage management is a procedure that allows users to maximize the utilization of storage devices while also protecting data integrity on whatever media on which it lives.
- The operating system is in charge of storing and accessing files.
- The creation of files, the creation of directories, the reading and writing of data from files and directories, as well as the copying of the contents of files and directories from one location to another are all included in storage management.

6. Security:

- The operating system uses password protection to protect user data and similar other techniques.
- It also prevents unauthorized access to programs and user data.
- The operating system provides various techniques which assure the integrity and confidentiality of user data.

7. Control Over System Performance:

- Operating systems play a pivotal role in controlling and optimizing system performance.
- They act as intermediaries between hardware and software, ensuring that computing resources are efficiently utilized.
- One fundamental aspect is resource allocation, where the OS allocates CPU time, memory, and I/O devices to different processes, striving to provide fair and optimal resource utilization.

8. Job Accounting:

- The operating system Keeps track of time and resources used by various tasks and users, this information can be used to track resource usage for a particular user or group of users.
- In a multitasking OS where multiple programs run simultaneously, the OS determines which applications should run in which order and how time should be allocated to each application.

9. Error-Detecting Aids:

- The operating system constantly monitors the system to detect errors and avoid malfunctioning computer systems.
- From time to time, the operating system checks the system for any external threat or malicious software activity.
- It also checks the hardware for any type of damage.

10. Coordination Between Other Software and Users:

- It directs and manages how different software programs can share your computer's resources without causing chaos.
- It ensures that when you want to use a program, it runs smoothly without crashing or causing problems for others.

11. Network Management:

- **Network Communication:** Operating systems help computers talk to each other and the internet. They manage how data is packaged and sent over the network, making sure it arrives safely and in the right order.
- **Settings and Monitoring:** They let you set up your network connections, like Wi-Fi or Ethernet, and keep an eye on how your network is doing. They make sure your computer is using the network efficiently and securely, like adjusting the speed of your internet or protecting your computer from online threats.

1.4. History of Operating System:

Generation	Time Period	Key Features	Examples
First Generation	1940s - Early 1950s	- No operating systems - Manual programming	ENIAC (No OS)
Second Generation	1950s - Early 1960s	-Simple batch processing - Introduction of punch cards	- IBM 7094 (Batch Processing)
Third Generation	1960s - Early 1970s	- Multiprogramming - Time-sharing systems	- IBM OS/360 - UNIVAC
Fourth Generation	1970s - 1980s	- GUI-based operating systems - Introduction of Unix	- MS-DOS - Unix
Fifth Generation	1980s - 1990s	- Advanced multitasking - Early mobile operating systems	- Windows 95 - Mac OS
Sixth Generation	1990s - 2000s	- Start of modern mobile OS - Open-source OS emerges	- Linux - Windows NT
Seventh Generation	2000s - Present	Cloud computing - Mobile-first OS - Virtualization - Integration of AI and machine learning	- Windows 10 - macOS - Android - iOS

1.5. Operating-System Structures:

1. Simple/Monolithic Structure:

- Monolithic Architecture is the oldest and the simplest type of Operating System Architecture.
- In this, each component is contained in a single kernel only.
- The various components in this OS Architecture communicate with each other via function calls.
- **Examples: Unix, Linux (early versions), MS-DOS.**

Advantages of Monolithic Architecture

- This type of architecture is easier to develop and maintain.
- It shows efficient performance because of the direct communication between the components.
- It is easy to secure as all the components are contained in a single file.

Disadvantages of Monolithic Architecture

- It is difficult to scalable.
- Hard to update since the change in one component result in an entire system update
- Large codebase which is difficult to understand.

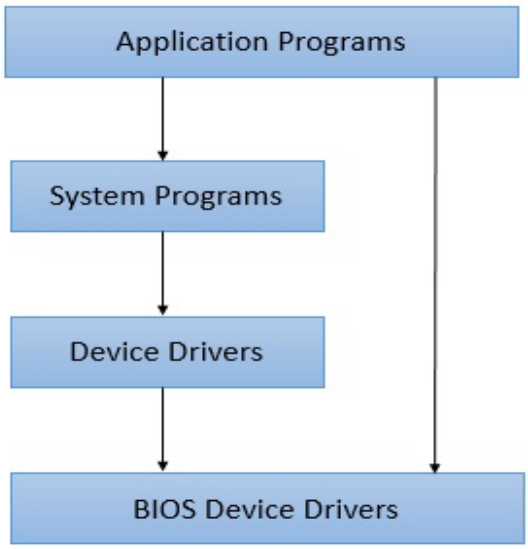


Fig: Monolithic /Simple Structure

2. **Microkernel Structure:**
   - As in case monolith architecture, there was single kernel, in micro-kernel, we have multiple kernels each one specialized service.
   - Each microkernel is developed independent to the other one and makes system more stable.
   - If one kernel fails, the operating system will keep working with other kernel's functionalities.
   - **Example: MacOS**

Advantages of Microkernel Architecture

- Easier to develop and maintain individual components
- Can support a wide range of operating systems and hardware platforms
- Enables customization and flexibility, allowing for the creation of tailored systems

Disadvantages of Microkernel Architecture

- Increased complexity due to the need to manage multiple components.
- Reduced performance due to communication overhead between components.
- Requires a significant amount of testing and validation to ensure correct operation of the system.

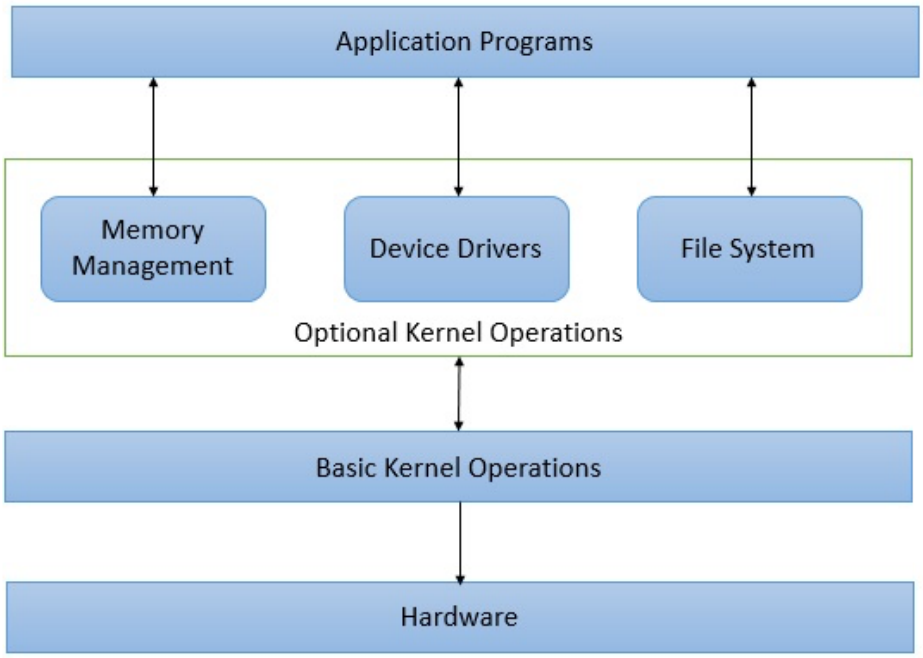
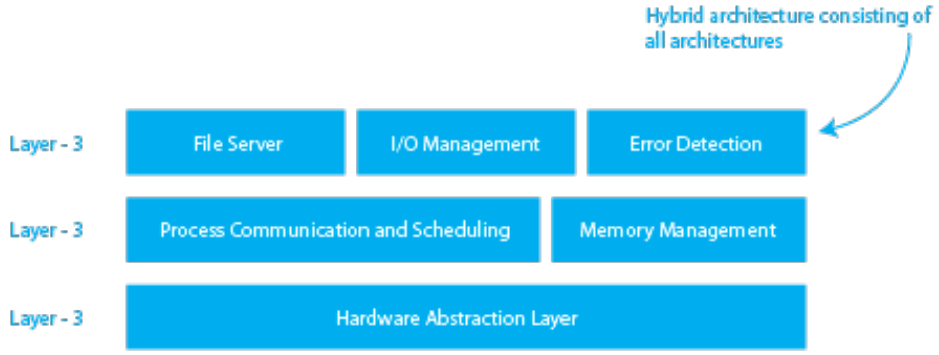


Fig: Microkernel Structure

3. **Hybrid Kernel Structure:**



- Hybrid-kernel structure is nothing but just a combination of both monolithic-kernel structure and micro-kernel structure.
- Basically, it combines properties of both monolithic and micro-kernel and make a more advance and helpful approach.
- **Example: MacOS, Windows OS.**

Advantages of Hybrid-Kernel Structure

- It offers good performance as it implements the advantages of both structure in it.
- It supports a wide range of hardware and applications.
- It provides better isolation and security by implementing micro-kernel approach.

Disadvantages of Hybrid-Kernel Structure

- Can be complex to design and maintain
- This architecture May result in slower system performance due to the increased overhead.
- Can be more prone to compatibility issues.

4. **Exo-Kernel Structure:**

- Exo-Kernal Architecture operating system was designed and developed at MIT.
- The aim of this design was to keep Kernel size minimal while allowing the application programs to manage hardware resources directly.
- The purpose of removing abstraction of operating system for hardware resources was to enable application programmer to write high performance code while Exo-kernel handles other operations.
- **Examples: Exokernel (developed at MIT), Nemesis OS**

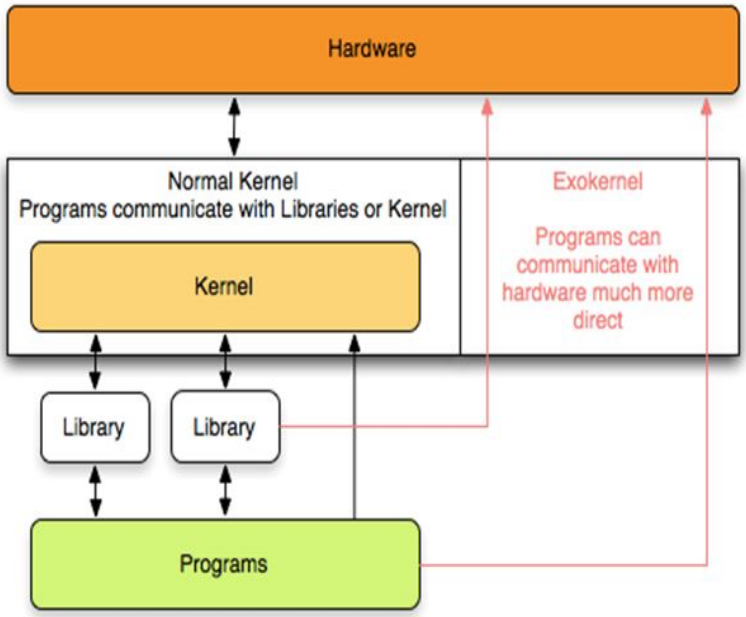


Fig: Exo-Kernel Structure

Advantages of Exo-Kernel

- Support for improved application control.
- Separates management from security.
- It improves the performance of the application.

Disadvantages of Exo-Kernel

- A decline in consistency.
- Exokernel interfaces have a complex architecture.

5. **Layered Structure:**

- In a layered architecture, the operating system is divided into layers, with each layer performing a specific set of functions.
- The layers are organized in a hierarchical order, with each layer depending on the layer below it.
- The layering approach makes the system easier to maintain and modify, as each layer can be modified independently without affecting the other layers.

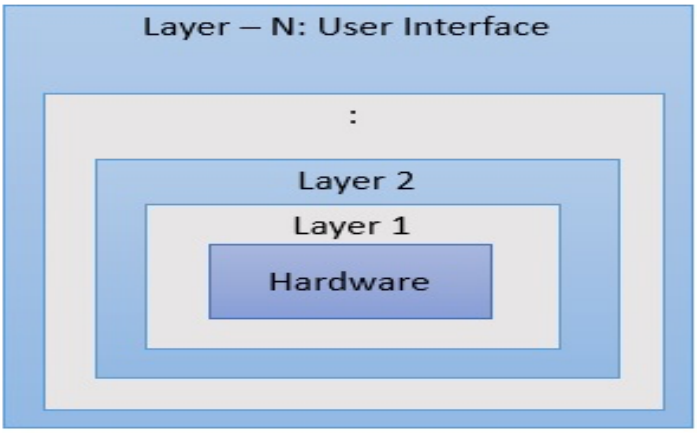


Fig: Layered Structure

Advantages of Layered Architecture

- Separation of concerns makes it easier to develop and maintain individual layers
- Components within a layer can be swapped out without affecting other layers
- Scalability is improved because layers can be scaled independently

Disadvantages of Layered Architecture

- Increased complexity due to the number of layers
- Slower performance due to communication overhead between layers
- Difficult to enforce strict separation between layers, which can lead to coupling and reduced modularity

Example: UNIX, Linux.

6. **Modular Structure:**

- Modular architecture operating system works on the similar principle as a monolith but with better design.
- A central kernel is responsible for all major operations of operating system.
- This kernel has set of core functionality and other services are loaded as modules dynamically to the kernel at boot time or at runtime.
- Example: Sun Solaris OS, Linux.

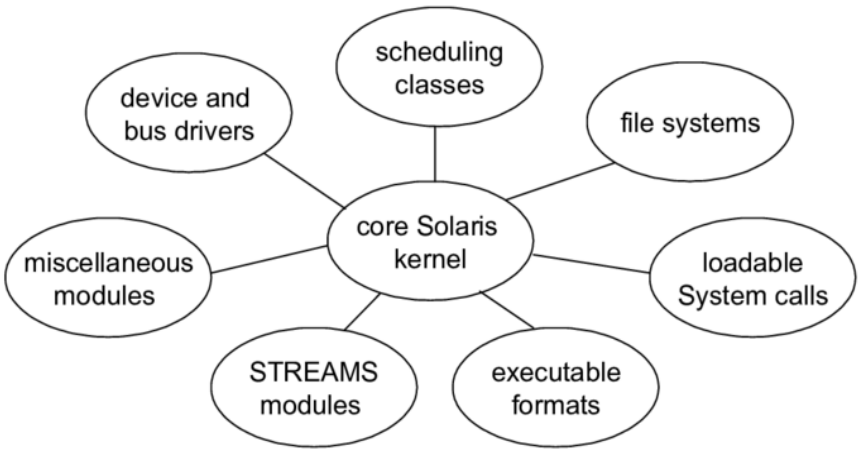


Fig: Modular Structure

Advantages:
- ✓ Customizable and flexible
- ✓ Easier to maintain and extend

Disadvantages:
- ✓ Can introduce performance overhead
- ✓ Complex designing

7. Client-Server Structure:

- ♦ In this structure, the OS is split into multiple services that communicate over a network.
- ♦ A client requests services, and a server responds and often used in distributed systems.

Advantages:
- ✓ Good for distributed systems
- ✓ Scalable and flexible.

Disadvantages:
- ✓ Network latency
- ✓ security issues

Examples: Distributed OS.

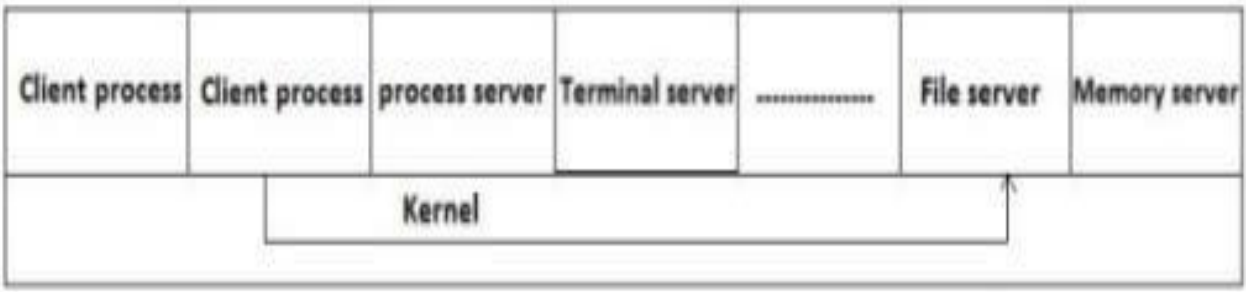


fig:- The client- server model

8. Virtual Machine Structure:

- ♦ In this kind of architecture, hardware like CPU, memory, hard disks are abstracted into virtual machines.
- ♦ User can use them with configure them using execution contexts.
- ♦ Virtual machine takes a good amount of disk space and is to be provisioned.
- ♦ Multiple virtual machines can be created on a single physical machine.

Advantages
- ✓ High Customizable
- ✓ Secure

Disadvantages
- ✓ Less Performing
- ✓ Complex designing

Examples: Linux

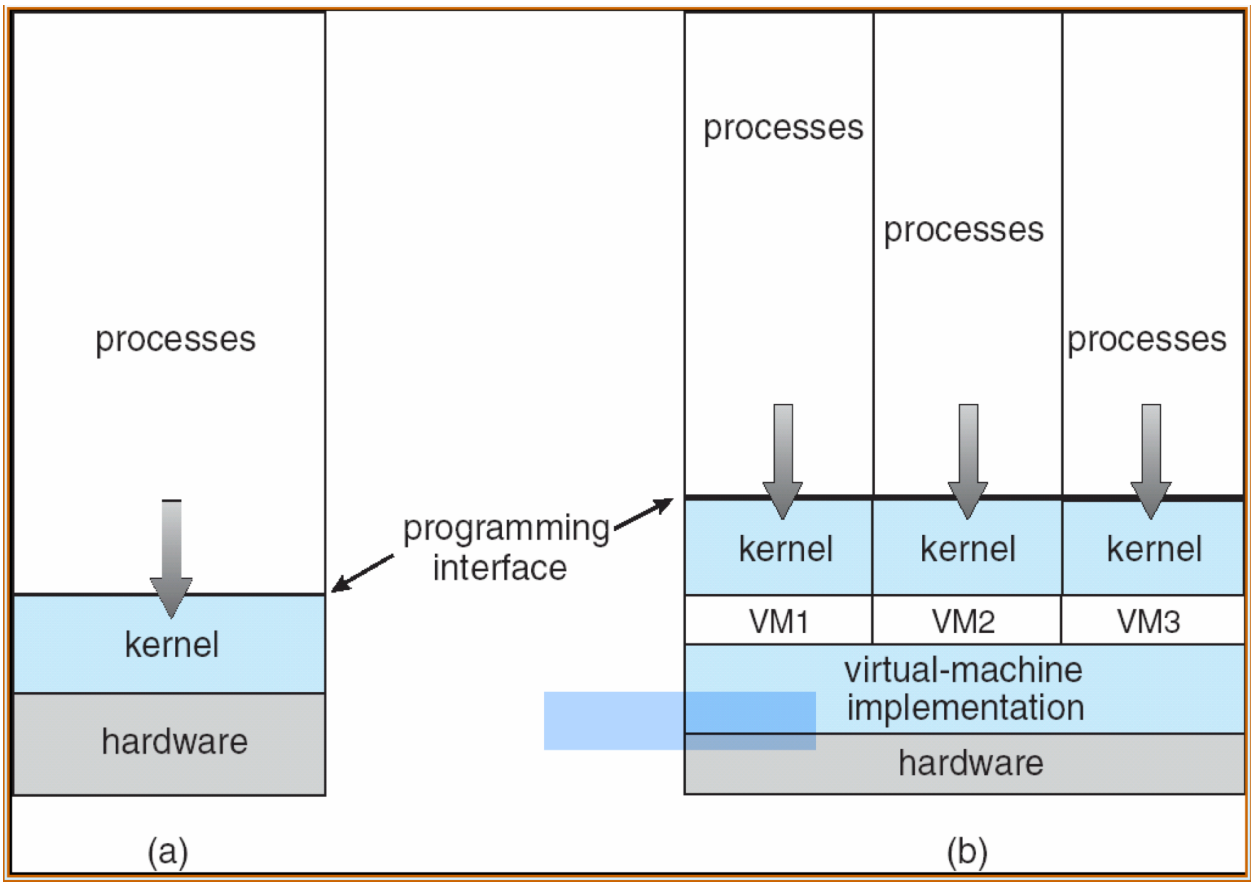


Fig: Virtual Machine Structure

1.6. Types of Operating Systems:

1. Batch Operating System:

- This type of operating system does not interact with the computer directly.
- There is an operator which takes similar jobs having the same requirements and groups them into batches.
- It is the responsibility of the operator to sort jobs with similar needs. Batch Operating System is designed to manage and execute many jobs efficiently by processing them in groups.

Advantages of Batch Operating System

- Multiple users can share the batch systems.
- The idle time for the batch system is very less.
- It is easy to manage large work repeatedly in batch systems.

Disadvantages of Batch Operating System

- Batch systems are hard to debug.
- It is sometimes costly.
- The other jobs will have to wait for an unknown time if any job fails.

Examples of Batch Operating Systems: Payroll Systems, Bank Statements, etc.

2. Multi-Programming Operating System:

- Multiprogramming Operating Systems can be simply illustrated as more than one program is present in the main memory and any one of them can be kept in execution.
- This is basically used for better utilization of resources.

Advantages of Multi-Programming Operating System

- Multi Programming increases the Throughput of the System.
- It helps in reducing the response time.

Disadvantages of Multi-Programming Operating System

- There is not any facility for user interaction of system resources with the system.
- Managing all processes and tasks can be a challenging task.

3. Multi-Processing Operating System:

- It is a type of Operating System in which more than one CPU is used for the execution of resources.
- It betters the throughput of the System.

Advantages of Multi-Processing Operating System

- It increases the throughput of the system.
- As it has several processors, so, if one processor fails, we can proceed with another processor.

Disadvantages of Multi-Processing Operating System

- Due to the multiple CPU, it can be more complex and somehow difficult to understand.
- It may require a bigger memory as multiple processors share the same main memory.
- The system crashes if the processor in charge of a particular task fails.

4. Multi-Tasking Operating System:

- Multitasking Operating System is simply a multiprogramming Operating System with having facility of a Round-Robin Scheduling Algorithm.
- It can run multiple programs simultaneously.

Advantages of Multi-Tasking Operating System

- Multiple Programs can be executed simultaneously in Multi-Tasking Operating System.
- It comes with proper memory management.

Disadvantages of Multi-Tasking Operating System

- The system gets heated in case of heavy programs multiple times.
- The system may run applications slowly due to the limited speed of its processors, and their response time may increase when processing multiple programs.
- The performance of the computer may be affected when several programs run simultaneously

5. Time-Sharing Operating Systems:

- Each task is given some time to execute so that all the tasks work smoothly.
- Each user gets the time of the CPU as they use a single system.
- These systems are also known as Multitasking Systems.

Advantages of Time-Sharing OS

- Each task gets an equal opportunity.
- Fewer chances of duplication of software.
- CPU idle time can be reduced.

Disadvantages of Time-Sharing OS

- Reliability problem.
- One must have to take care of the security and integrity of user programs and data.
- Data communication problem.

6. Distributed Operating System:

- The Distributed Operating system is not installed on a single machine, it is divided into parts, and these parts are loaded on different machines.
- A part of the distributed Operating system is installed on each machine to make their communication possible.
- Distributed Operating systems are much more complex, large, and sophisticated than Network operating systems because they also have to take care of varying networking protocols.

Advantages of Distributed Operating System

- Failure of one will not affect the other network communication, as all systems are independent of each other.
- Since resources are being shared, computation is highly fast and durable.
- Load on host computer reduces.

Disadvantages of Distributed Operating System

- Failure of the main network will stop the entire communication.
- To establish distributed systems the language is used not well-defined yet.

7. **Network Operating System**:

- An Operating system, which includes software and associated protocols to communicate with other computers via a network conveniently and cost-effectively, is called Network Operating System.

Advantages of Network Operating System

- Highly stable centralized servers.
- Security concerns are handled through servers.
- New technologies and hardware up-gradation are easily integrated into the system.

Disadvantages of Network Operating System

- Servers are costly.
- User has to depend on a central location for most operations.
- Maintenance and updates are required regularly.

8. Real-Time Operating System:

- In Real-Time Systems, each job carries a certain deadline within which the job is supposed to be completed, otherwise, the huge loss will be there, or even if the result is produced, it will be completely useless.
- Real-time systems are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.

Advantages of RTOS

- Maximum Consumption: Maximum utilization of devices and systems, thus more output from all the resources.
- Task Shifting
- Focus on running applications and less importance on applications that are in the queue.

Disadvantages of RTOS

- Very few tasks run at the same time and their concentration is very less on a few applications to avoid errors.
- Use heavy system resources: Sometimes the system resources are not so good and they are expensive as well.
- The algorithms are very complex and difficult for the designer to write on.

9. Embedded System OS:

- An embedded operating system is a computer operating system designed for use in embedded computer systems.
- It has limited features. The term "embedded operating system" also refers to a "real-time operating system".
- The main goal of designing an embedded OS is to perform specified tasks for non-computer devices.
- It allows the executing programming codes that deliver access to devices to complete their jobs.

Advantages

- It is small in size and faster to load.
- It is low cost.
- It is easy to manage.

Disadvantages

- It has limited resources for memory.
- It isn't easy to take a back of embedded files.
- It's hardware is limited.

1.7. Kernel Data Structures:

1.7.1. Introduction to Kernel:

- Kernel is a computer program that is a core or heart of an operating system.
- Kernel is the core part of an OS(Operating system); hence it has full control over everything in the system.
- Each operation of hardware and software is managed and administrated by the kernel.
- It acts as a bridge between applications and data processing done at the hardware level.
- It is the part of the OS that always resides in computer memory and enables the communication between software and hardware components.
- It manages system resources such as memory, CPU, and input/output devices, and provides a layer of abstraction between the hardware and higher-level software components.

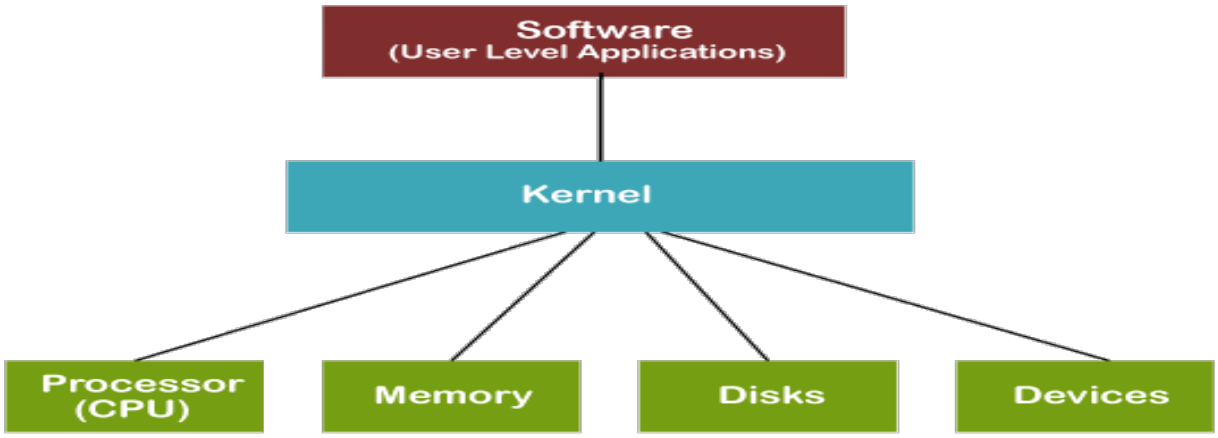


Fig: Kernel in OS

1.7.2. **Types of Kernels:**

1. **Monolithic Kernel:**
   - In a monolithic kernel, all the operating system services run in kernel space, which provides efficient communication between components.
   - However, it also means that a failure in one component can crash the entire system.
   - Examples :Linux, Solaris

Advantages

- Faster process execution with no separate user space and kernel space
- Smaller source and compiled forms

Disadvantages

- A failure in one component can crash the entire system
- Not portable, must be rewritten for new architectures

2. **Microkernel:**
   - A microkernel only includes the essential services in kernel space, and other services run in user space.
   - This approach reduces the kernel's size and improves the system's reliability.
   - Examples: QNX, MINIX, L4

Advantages

- New services can be added without modifying the whole OS Kernel process crashes do not necessarily cause the whole system to crash

Disadvantages

- More software is required for interfacing, which can reduce system performance
- Complicated process management
- Difficult to fix messaging bugs

3. **Hybrid Kernel:**
   - A hybrid kernel is a combination of monolithic and microkernel architectures.
   - It includes a small kernel in kernel space, with other services running in user space. This approach provides both efficiency and reliability.
   - Examples: Windows NT (XP, Vista, 7, 8, 10), macOS (versions before 11.0)

Advantages

- No need to reboot the system for testing.
- Faster integration of third-party technologies.

Disadvantages

- Increased chances of bugs due to multiple interfaces.
- Maintenance of modules can be difficult for some administrators, especially when dealing with symbol differences.

4. **Nanokernel:**

- A nanokernel is a minimalistic operating system architecture where the kernel provides only the most basic functions needed for an operating system to run.
- All other functionality, including device drivers, file systems, and network protocols, are implemented as user-level processes.
- Examples: QNX, MINIX

Advantages

- Small footprint and low overhead, leading to high performance.
- High modularity and flexibility, as only the most basic functionality, is provided by the kernel, allowing for more customization.

Disadvantages

- More complex system design, as user-level processes need to be carefully designed and managed.

5. **Exokernel:**

- An exokernel is an operating system architecture that exposes the hardware resources directly to user-level applications; while providing minimal services, such as address space management and protection.
- It allows user-level applications to directly control hardware resources, such as the CPU, memory, and network interfaces.
- Examples: Xok/ExOS, Nemesis, SPIN.

Advantages

- High performance and low overhead, as user-level applications, can directly control hardware resources.
- Flexibility, as application developers, can customize their own resource management strategies.

Disadvantages

- In complex system design, the kernel has to provide fine-grained control over hardware resources to user-level applications.

1.7.3. **Function of Kernel:**

The function of Kernel includes:

1. Process Management

- Scheduling and execution of processes.
- Context switching between processes.

2. Memory Management
- Allocation and deallocation of memory space.
- Managing virtual memory.

3. Device Management
- Managing input/output devices.
- Providing a unified interface for hardware devices.

4. File System Management
- Managing file operations and storage.
- Handling file system mounting and unmounting.

5. Resource Management
- Managing system resources (CPU time, disk space, network bandwidth)
- Allocating and deallocating resources as needed

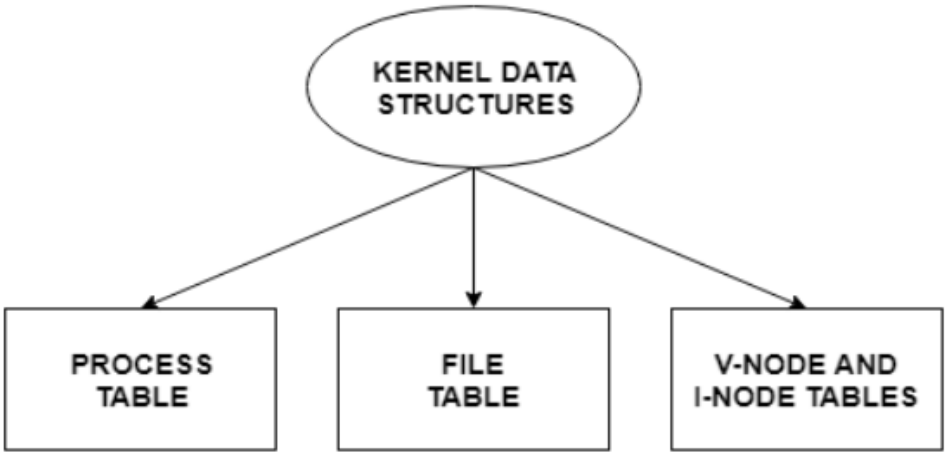
6. Security and Access Control
- Enforcing access control policies.
- Managing user permissions and authentication.

7. Inter-Process Communication
- Facilitating communication between processes.
- Providing mechanisms like message passing and shared memory.

1.7.4. **Objectives of Kernel:**
- To establish communication between user-level applications and hardware.
- To decide the state of incoming processes.
- To control disk management.
- To control memory management.
- To control task management.

1.7.5. **The kernel data structures**:
- They are very important as they store data about the current state of the system.
- For example, if a new process is created in the system, a kernel data structure is created that contains the details about the process.

Kernel Components



Process Table

- The process table stores information about all the processes running in the system.
- These include the storage information, execution status, file information etc.
- When a process forks a child, its entry in the process table is duplicated including the file information and file pointers. So, the parent and the child process share a file.

File Table

- The file table contains entries about all the files in the system.
- If two or more processes use the same file, then they contain the same file information and the file descriptor number.
- The file table also contains v-node and i-node pointers which point to the virtual node and index node respectively.
- These nodes contain information on how to read a file.

V-Node and I-Node Tables

- Both the v-node and i-node are references to the storage system of the file and the storage mechanisms.
- They connect the hardware to the software.
- The v-node is an abstract concept that defines the method to access file data without worrying about the actual structure of the system.
- The i-node specifies file access information like file storage device, read/write procedures etc.

1.8. System Interface:

- System Interface defines the mechanisms through which applications interact with the kernel.
- It provides a well-defined set of functions and protocols that applications can use to request kernel services.

Key components of the system interface include:

System Calls: Functions that applications invoke to request kernel services.

Application Programming Interface (API): A set of libraries and functions that provide a higher-level abstraction of system calls.

Device Drivers: Provide a standardized interface for applications to interact with hardware devices.

1.9. System Calls:

- A system call is a method for a computer program to request a service from the kernel of the OS on which it is running.
- A system call is a method of interacting with the operating system via programs.
- A system call is a request from computer software to an operating system's kernel.

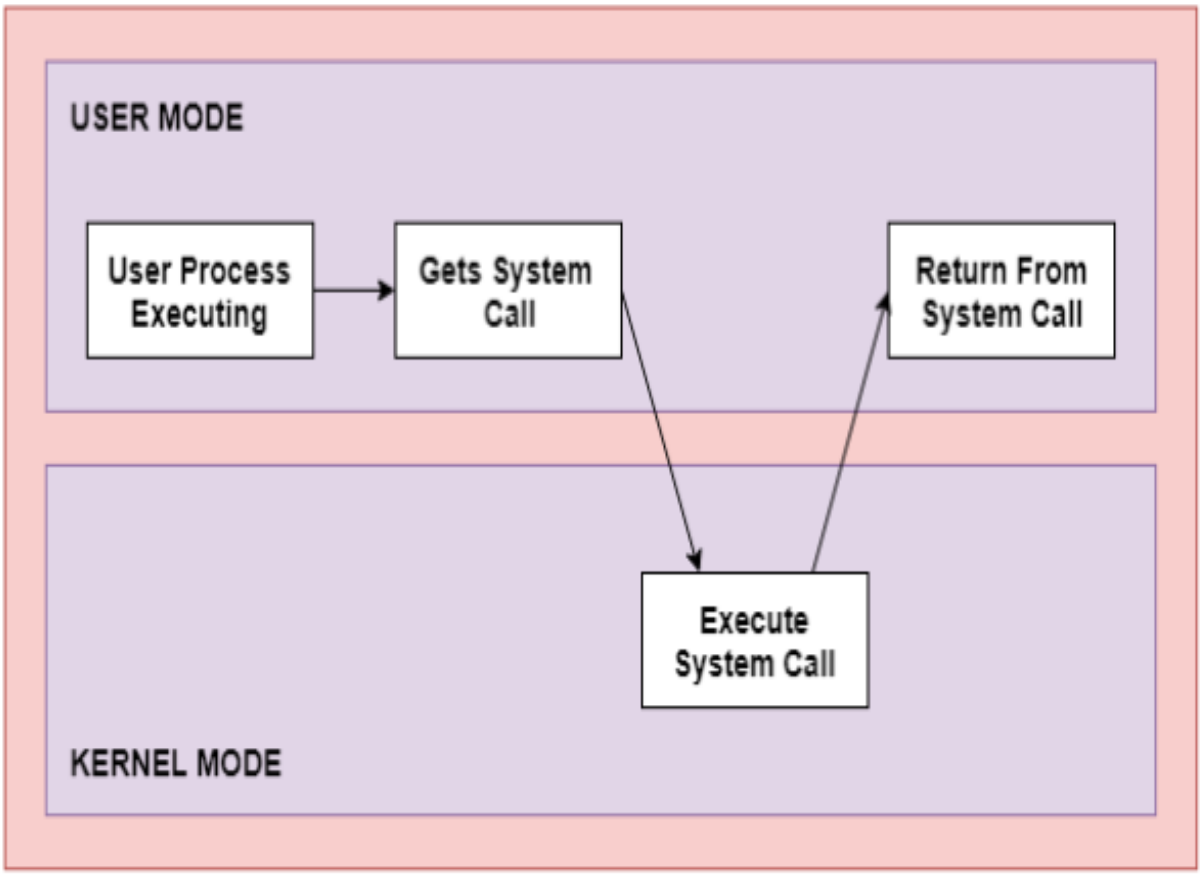


Fig: Process of System Call

1.9.1. Need of system calls in Operating System:

There are various situations where you must require system calls in the operating system. Following of the situations are as follows:

1. It must require when a file system wants to create or delete a file.
2. Network connections require the system calls to sending and receiving data packets.
3. If you want to read or write a file, you need to system calls.
4. If you want to access hardware devices, including a printer, scanner, you need a system call.
5. System calls are used to create and manage new processes.

1.9.2. Types of System Calls:

1. **Process Control:**

- Process control is the system call that is used to direct the processes.
- Some process control examples include creating, load, abort, end, execute, process, terminate the process, etc.

2. **File Management:**

- File management is a system call that is used to handle the files.
- Some file management examples include creating files, delete files, open, close, read, write, etc.

3. **Device Management:**

- Device management is a system call that is used to deal with devices.
- Some examples of device management include read, device, write, get device attributes, release device, etc.

4. **Information Maintenance:**

- Information maintenance is a system call that is used to maintain information.
- There are some examples of information maintenance, including getting system data, set time or date, get time or date, set system data, etc.

5. **Communication:**

- Communication is a system call that is used for communication.
- There are some examples of communication, including create, delete communication connections, send, receive messages, etc.

1.9.3. How does System Call Work?

Here is a detailed explanation step by step how system calls work:

- **Users need special resources:** Sometimes programs need to do some special things that can't be done without the permission of the OS like reading from a file, writing to a file, getting any information from the hardware, or requesting a space in memory.
- **The program makes a system call request:** There are special predefined instructions to make a request to the operating system. These instructions are nothing but just a "system call". The program uses these system calls in its code when needed.
- **Operating system sees the system call:** When the OS sees the system call then it recognizes that the program needs help at this time so it temporarily stops the program execution and gives all the control to a special part of itself called 'Kernel'. Now 'Kernel' solves the need of the program.
- **The operating system performs the operations:** Now the operating system performs the operation that is requested by the program. Example: reading content from a file etc.
- **Operating system give control back to the program:** After performing the special operation, OS give control back to the program for further execution of program.